

Computer graphics coursework 1 report

Introduction

This report covers a set of programming tasks focused on basic 2D graphics. Each section explains and demonstrates techniques like drawing lines, rotating objects, drawing triangles, and blitting images. The other tasks (test and benchmark) are to evaluate the performance of the system.

1.1 Setting pixels

To set individual pixels, it is important to understand how the system stores them in memory. The image data is held in a one-dimensional array where pixels are ordered from top right to bottom left. Each pixel uses four slots in the array: three for the red, green, and blue values, and one padding slot to make the pixel 32 bits in total.



Figure1: drawn pixel background (stars)

The ***get_linear_index()*** function converts the two-dimensional X and Y coordinates of a pixel into its corresponding linear index, starting from position 0. This is done by first calculating the vertical row (Y-position) by multiplying the Y value by the width of the surface, which represents one full row of pixels. The X value is then added to offset the position horizontally, resulting in the formula $Y * \text{width} + X = \text{linear index}$. The ***set_pixel_rgb()*** function then assigns the colour values at that position, placing red at index 0, green at 1, blue at 2, and leaving the padding slot set to 0.

1.2 Clipping lines

The line clipping process begins by checking whether the entire line lies outside the screen boundaries. This is done using a logical “and” condition to test all four sides of the screen. If the line is completely outside any of these sides, the function returns false and no line is drawn. However, if part of the line remains within the visible area, the endpoints (*aBegin/ aEnd*) are adjusted based on which side extends beyond the boundary. The affected endpoint, either the beginning or the end of the line, is set to the maximum value allowed for that boundary. In this case, the function returns true, allowing the visible portion of the line to be drawn.

1.3 Drawing lines

I used the Digital Differential Analyzer (DDA) algorithm¹ to draw lines by calculating all the points in between two points. The algorithm begins by finding the differences in the X and Y coordinates. The total number of steps is determined by taking the larger of these two differences. The increments for X and Y are then calculated by dividing each difference by the total number of steps. A for loop is used to draw the pixels, starting from the initial coordinates. Each pixel position is rounded to the nearest integer, and a bounds check ensures that only pixels within the surface are drawn. After each iteration, the X and Y coordinates are updated by adding their respective increments to move to the next pixel along the line. *Note: the algorithm is of $O(n)$ complexity.*

1. $\Delta x = x_2 - x_1, \Delta y = y_2 - y_1$: The horizontal and vertical differences (change in x and y)
2. **steps** = $\max(\Delta x, \Delta y)$; : Total number of steps (pixels) to draw
2. $x_{\text{inc}} = \frac{\Delta x}{\text{steps}}, y_{\text{inc}} = \frac{\Delta y}{\text{steps}}$: The x and y increments for each step
3. $x_{\text{inc}} = \frac{\Delta x}{\text{steps}}, y_{\text{inc}} = \frac{\Delta y}{\text{steps}}$: The x and y increments for each step
4. $(x_{k+1}, y_{k+1}) = (x_k + x_{\text{inc}}, y_k + y_{\text{inc}})$: Formula to calculate the next pixel position

Figure 2: DDA algorithm mathematical expressions

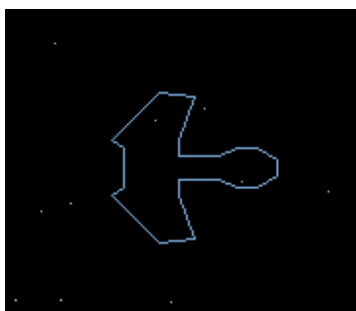


Figure 3: Drawn spaceship

1.4 2D Rotation

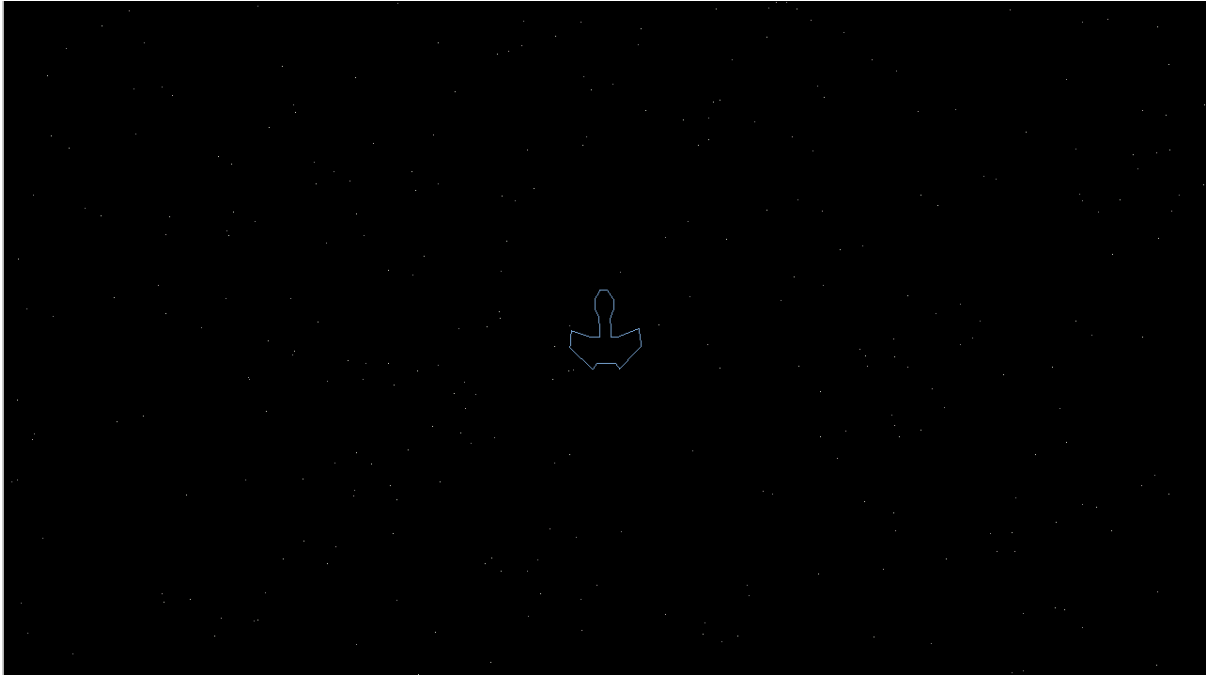


Figure 4: rotated spaceship

1.5 Drawing triangles

The triangle is rasterized using the barycentric coordinate rasterization method. The process starts by defining an edge function that determines whether a point lies inside, on the edge, or outside the triangle based on its position relative to the three sides. The algorithm then checks for degenerate triangles, which have an invalid area, and determines the orientation of the triangle. Next, the smallest rectangle that fully contains the triangle is calculated and clipped to the surface boundaries to ensure that no pixels outside the visible area are processed. The main loop then iterates through every pixel within this rectangle, using the edge function to test each point. If the point lies inside the triangle, the barycentric coordinates are calculated and normalized to determine the pixel's final colour. This method ensures efficient handling of cases where the shape is partially outside the surface.

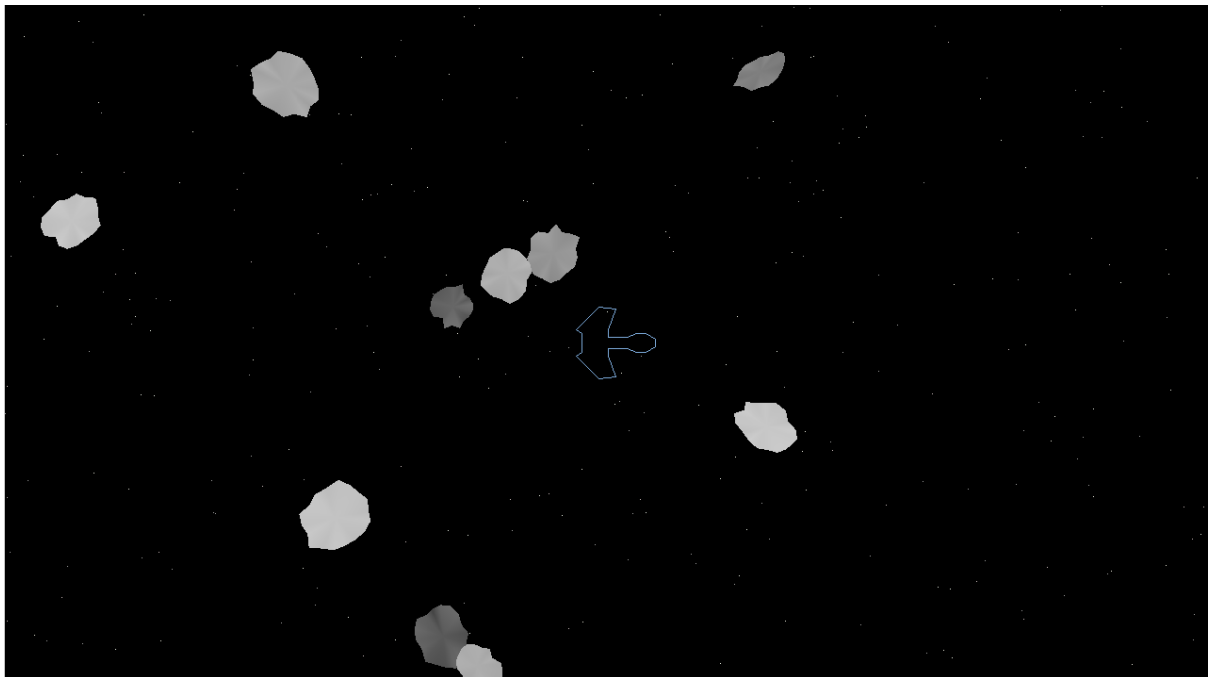


Figure 5: Drawn asteroid field

1.6 Blitting Images

The process of blitting involves copying an image file onto a pixel surface by transferring each pixel individually. This is like drawing individual pixels, where the *get pixel* function retrieves the colour of a pixel by first calculating its linear index. Using this index, the function returns the **ColorU8_sRGB_Alpha** value of the specific pixel. In the *Blit_masked()* function, rectangular clippingⁱⁱ is applied to optimize performance. This approach replaces an earlier per-pixel clipping method, which caused significant overhead on both the CPU and main memory.

Aspect	Before (Per-Pixel Clipping)	After (Rectangular Clipping)
Bounds Checking	Checks every pixel to see if it's on screen	Figures out visible area once before starting
Pixels Processed	Processes every pixel in the image, even if off-screen	Only processes pixels that are actually visible
Loop Iterations	Loops through the entire image size	Loops only through the visible portion
Branch Predictions	Makes a decision for every single pixel	No decisions needed inside the main loop
Early Exit	Always processes everything	Skips entirely if the image is completely off-screen
Memory Access Pattern	May read from memory locations that won't be used	Reads memory in a smooth, continuous way
Computational Complexity	Work increases with full image size	Work increases only with visible portion size

Figure 6: Table comparing per-pixel clipping against rectangular clipping

The algorithm begins by obtaining the dimensions of the image and the surface. Rectangular clipping then determines which parts of the image lie within the visible area and excludes any pixels outside the surface before the main loop begins. This avoids unnecessary checks for every pixel. The loop then iterates over the clipped width and height of the image, adjusting the coordinates to map to the image's pixel data. Each pixel colour is retrieved and blended with the black background using alpha blending, after which the pixel is drawn onto the surface with the proper coordinate offset.

1.7 Testing lines

The first test focuses on partially off-screen lines, covering all four sides and surface corners with lines drawn outside-to-inside and inside-to-outside. It uses assertions inside `set_pixel_rgb()` to prevent drawing outside the surface. Verification checks for pixel continuity along rows or columns based on the line direction, confirming correct drawing.

The second test checks for one-pixel-thin lines (horizontal, vertical, steep, and shallow diagonals) with no gaps. Horizontal lines are checked for column thickness and length, and vertical lines for row thickness. For diagonals, if X and Y change at the same rate, both rows and columns are verified; otherwise, column thickness is checked for shallow diagonals and row thickness for steep diagonals.

The third test verifies that lines drawn in reverse order produce the same result, including those partially off-screen. Two surfaces are used (one normal line, one reversed line), and their pixels are compared one by one to ensure they are identical.

The final test ensures two connected lines form a longer line without gaps. To confirm continuity, the algorithm checks each pixel's eight neighbours. A valid continuous line must have exactly two pixels with one neighbour, no isolated pixels, and no pixels with more than two neighbours. Meeting these conditions confirms the line is continuous and gap-free.

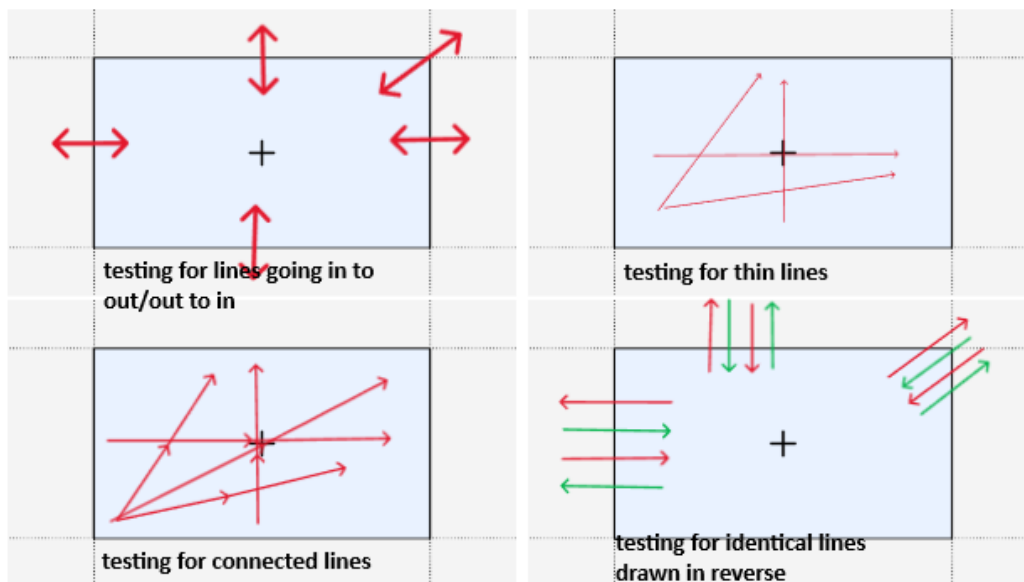


Figure 7: Depiction of different line tests

1.8 Testing triangles

Before running the tests, two helper functions are defined. The first, *find_most_red_pixel()*, scans every pixel in the surface and returns the pixel with the highest red value. The second, *find_least_red_nonzero_pixel()*, also scans the surface but ignores black pixels and returns the pixel with the lowest red value that is not zero.

The first test checks whether two triangles placed together form a seamless rectangle. The two triangles are drawn in grey to form the rectangle shape. After drawing, the test uses the two helper functions. The value from *find_most_red_pixel()* confirms that pixels were drawn. The value from *find_least_red_nonzero_pixel()* is then used to check for gaps along the shared edge. If this value is zero or below, it indicates that at least one pixel was missed, meaning the triangles did not fully connect. A correct result shows no missing pixels and therefore no seam.

The second test verifies that drawing a triangle with different edge orders (for example, drawing the vertices in clockwise vs counterclockwise order) produces the exact same result. Two surfaces are used here: one with the triangle drawn in the original order and the other drawn in the reversed order. The test passes if every pixel on both surfaces is identical, confirming that the algorithm behaves consistently regardless of edge ordering.

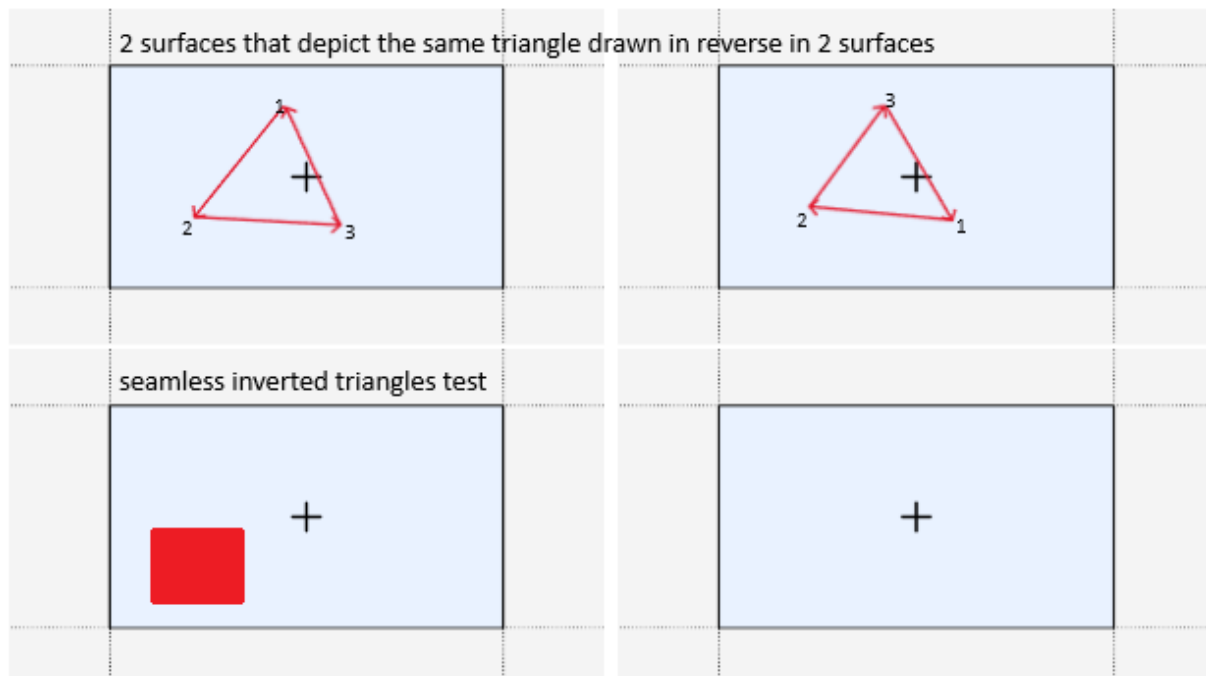


Figure 8: Depiction of different triangle tests

1.9 Benchmarking specifications

System 1

CPU model name	AMD Ryzen 7 9700X 8-Core processor
L1 data cache amount (core)	640 KB
L2 cache amount (core)	8.0 MB
System RAM amount	32 GB
System RAM type and speed	DDR5 6000MT/s
Operating system	Windows 11 Pro
Compiler	gcc (Ubuntu 14.2.0-4ubuntu2~24.04) 14.2.0

System 2

CPU model name	12th Gen Intel® Core™ i7-12700 × 20
L1 data cache amount (core)	512 KiB
L2 cache amount (core)	12 MiB
System RAM amount	32 GB
System RAM type and speed	DDR4 3200 MT/s
Operating system	Red Hat Enterprise Linux 9.6 (Plow)

1.10 Benchmark: lines I

The first variable tested is **line length**. This measurement depends directly on the DDA algorithm, which has $O(n)$ time complexity, so the expected result was a linear increase in execution time as line length increased. The actual results were faster than expected due to **cache efficiency**, showing about a **32% decrease in CPU time** compared to the projected values. For example, the second data point ranged from **5753 cycles for a 10-pixel line** to **34342 cycles for a 100-pixel line**, which is better than purely linear growth. This effect was especially noticeable for **horizontal lines**, since they benefit from **sequential memory writes**.

The second variable is the **number of lines drawn**. This does not change the line-drawing algorithm itself but simply scales how many times it is executed. Because of this, I expected the performance to also show **linear behavior** with $O(n)$ complexity. The results matched this prediction closely, with only **about 5% variance** across the data.

The final variable is the **line color**. Changing the color does not affect the logic or loop of the DDA algorithm and adds only a very small overhead. I expected the influence of color to be minimal, with at most 5% difference in performance. The results confirm this, with the **largest variance being 4.7%**, which is effectively negligible.

(benchmarks ran on system 1)

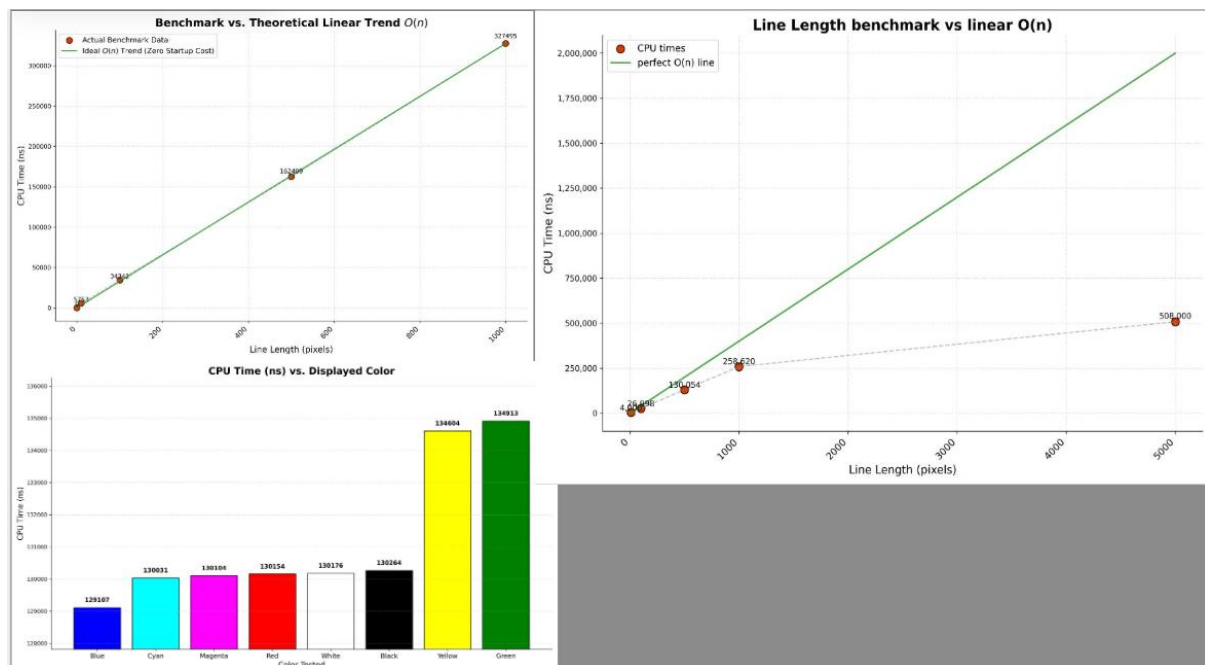


Figure 9: line benchmarking graphs

1.11 Benchmarking Blitting

The first variable tested across the three blitting methods is **resolution**. Since `blit_masked()` does **alpha blending** on every pixel, it should be the slowest, while all three methods still scale as **O(n)**. So, the expected order was `blit_ex_memcpy()` fastest (it uses an optimized `memcpy` to copy full rows), then `blit_ex_solid()`, and finally `blit_masked()` as the slowest. The results confirmed this, showing that alpha blending has a clear cost. However, at higher resolutions the performance curves **flatten out**. At that point, **memory bandwidth** becomes the main limit and **cache effects matter less**. Because of this, the speed difference between the three methods **shrinks** at large resolutions, even though `blit_masked()` remains the slowest.

For the second variable my hypothesis is that **runtime will scale linearly** with **width x height** of the image, having a plateau on **higher image sizes** due to **cache miss issue not being a case**. The **actual results** show that the **scaling only happens at a rate of 1.4-1.5** this suggests **cache efficiency** and **memory access** works particularly well for the algorithms with the slight deviation of the `blit_masked()`.

Finally, for **clipping** we expect that the **performance will increase** as **less pixels are drawn**, we expect that the **image clipping** will have a **higher than directly proportional** on the performance since removing pixels to be drawn takes away the **biggest overhead**. The **result proves the hypothesis right** and even goes further beyond in impacting performance by an **80-85% reduction in proportional CPU times** this is likely because **less pixels reduce cache misses** by a large margin and **utilises the CPU pipeline better**. (Benchmarks ran on System 1)

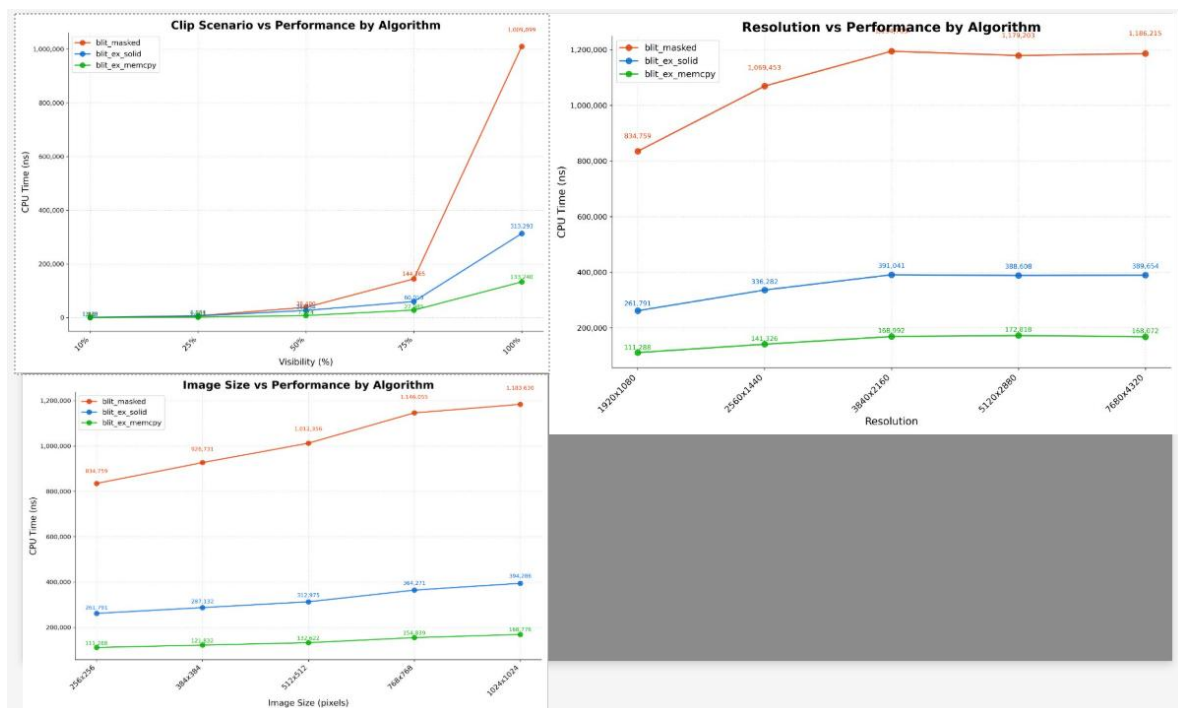


figure 10: Blitting benchmark graphs

1.12 Benchmarking Lines II

In this section I compared my implemented **DDA algorithm** with **Bresenham's lineⁱⁱⁱ drawing algorithm**. Both are of **$O(n)$ time complexon**.

I started by comparing them **horizontally** first this resulted on the `draw_ex_line_solid()` algorithm (**Bresenham's**) running **much quicker**. My hypothesis is because **Bresenham's uses integer operations** instead of **floating-point operations** for DDA; this causes **Bresenham's algorithm to allow the CPU pipeline to run more smoothly**, it also means **more efficient use of cache**.

Next, I tested them **diagonally**, for this I used the **provided baseline algorithm** (runs in **$O(n)$**), the same result is achieved from line length **0-1000 (integer faster than floating point)** however at **length 5000 (when clipping is needed)** **DDA significantly outperforms Bresenham's algorithm**. My hypothesis is that this happens due to **Bresenham's algorithm being implemented here does not call the `clip_line()` algorithm to ignore irrelevant pixels** if that were the case Bresenham's would outperform DDA.

Finally, the **baseline algorithm is the quickest** since it does not introduce any logic however because it also **doesn't call the `clip_line()` algorithm** it forces the program to try and draw a pixel which results in a **program crash**.

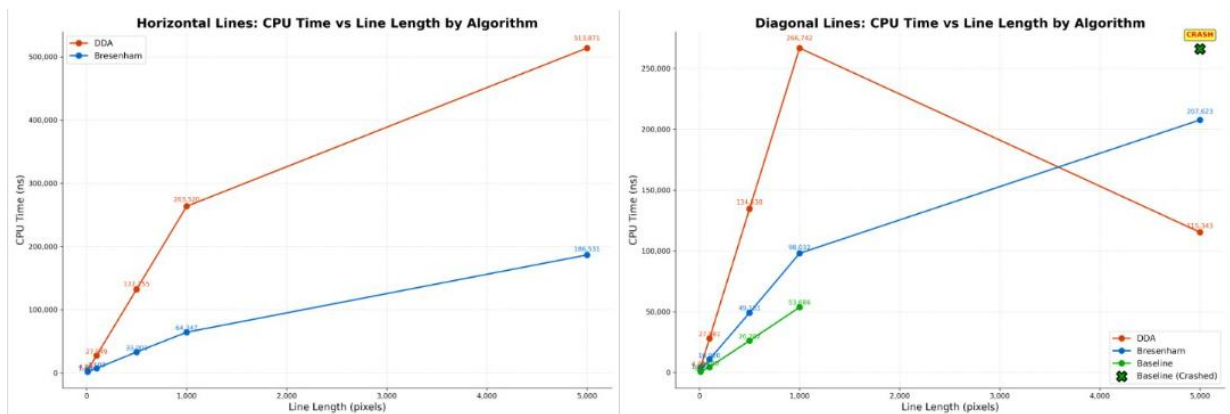


Figure 11: line algorithms benchmark graphs

1.13 Your own spaceship

My spaceship depicts the millennium falcon from star-wars (attempted to my best degree), it uses 19 plotted points using an octagon for the main body and individual points for the rest.

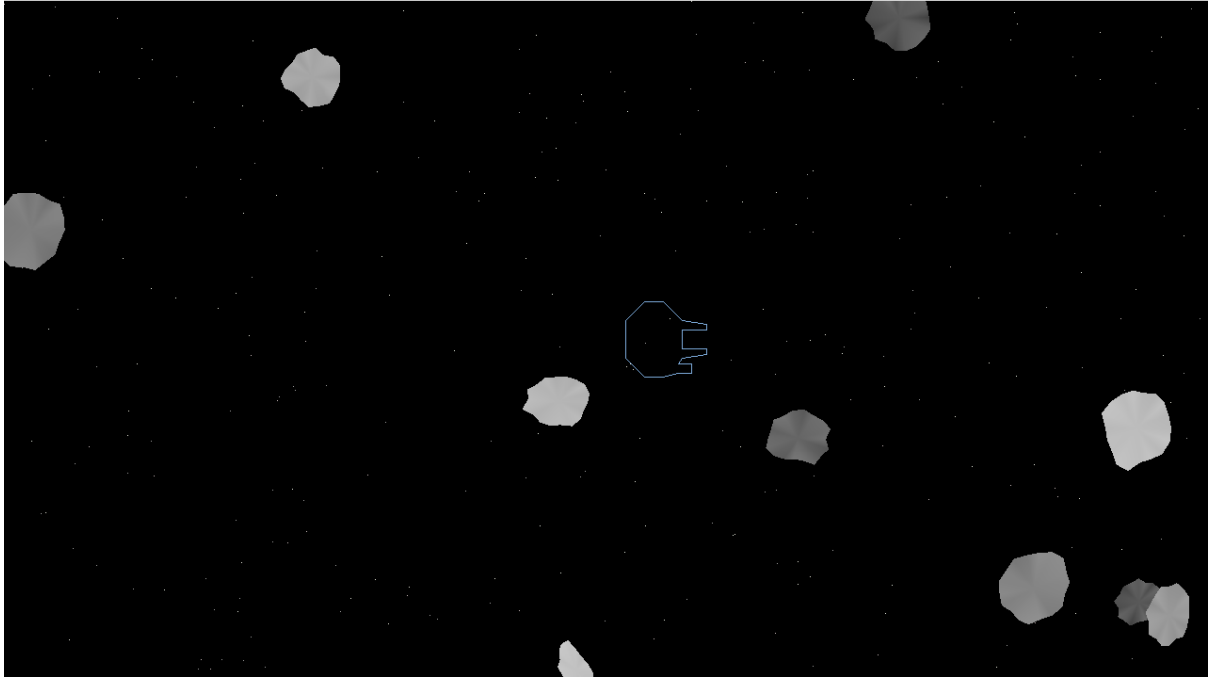


Figure 12: My own depiction of the millennium falcon

ⁱ Huddar, M., 2021. 1. Solved Digital Differential Analyzer | DDA Algorithm | Line Drawing Algorithm | Computer Graphics and Visualization [video online]. YouTube. Available at: <https://www.youtube.com/watch?v=C2YrUzTHOyo> [Accessed 7 Nov 2025].

ⁱⁱ **The Coding Train, 2017.** Coding Challenge #146: Image Cropping / Clipping [video online]. YouTube. Available at: <https://www.youtube.com/watch?v=5uH8L1FRevs> [Accessed 7 Nov 2025]

ⁱⁱⁱ **Neso Academy, 2018.** Bresenham's Line Drawing Algorithm [video online]. YouTube. Available at: <https://www.youtube.com/watch?v=IDFB5CDpLDE> [Accessed 7 Nov 2025].